

COMP1531: Software Engineering Fundamentals

L. Cheung

August 4, 2026

Contents

1	Introduction	2
1.1	Course Structure	2
1.2	Introduction to Software Engineering	2
1.3	Common Software Development Models	3
1.4	JavaScript	3
1.4.1	Data Structures	3
2	Git	5
3	Unigotchi - Iteration 0	6
4	Importing Libraries	7
5	asdf	8
6	Dynamic Verification	9
6.0.1	Coverage	9
7	Static Verification	10
8	Continuous Integration	11
9	Advanced Functions	12
9.1	Map, Reduce, Filter	13
10	HTTP Servers	14
11	Persistence	15
12	Exceptions	16
13	Authentication and Authorisation	17
14	Session	18
15	Modelling	19
16	Software Complexity	20
17	Agile Software Engineering	21
17.1	Validation	22
17.2	Deployment	22

Chapter 1

Introduction

1.1 Course Structure

- Lab tasks are worth 10% in total
- Group project worth 45% of grade
- The final exam is worth 45%

1.2 Introduction to Software Engineering

What is the difference between Computer Science and Software Engineering.

- Computer science focuses on understanding how computers work and function
- Software engineering is about creating software for people to use, and building software that is maintainable over time

The Software Development Life Cycle is a structured approach to software development that includes:

- Requirements analysis
 - Talk to potential clients
 - Work out what is needed of the software
- Design
 - Designing an outline for the software
- Development
 - Writing the actual code to handle the task
- Testing
 - Test the program to see if it meets the initial requirements
- Deployment
 - Deploy the software to users
- Maintenance
 - Continue to develop, fixing bugs and implementing new features

Software quality and productivity is supported by three key pillars: people, technology, and processes.

1.3 Common Software Development Models

- Waterfall → linear and structured
- Prototyping → get early feedback
- Iterative development → Continuous improvement and feature addition
- Agile → Iterative development but getting feedback on each feature after it is completed

1.4 JavaScript

- Developed in 1995 so Netscape could add scripting to their browser
- Extensively used by large tech companies
- Front-end software
- NodeJS compiles and runs JavaScript code
- NodeJS is interpreted, ie. a high level language. The code gets interpreted before it gets compiled into machine language.
- JS type shenanigans
- `let` creates a variable that can be reassigned
- `const` creates a variable that cannot be reassigned. However, objects can still be manipulated.
- `${variable}` can be used to insert the value of a variable into a string
- Strings
 - String concatenation can be performed using `+`, eg. `"some text" + variable`
 - Adding variables into strings can be done using string interpolation, eg. ``Hello, my name is ${name}`` (where `name` is a variable)
- Control structures
 - Very similar syntax to C, nothing important to note

1.4.1 Data Structures

- Data structures are designed to scale complex tasks
- Two important data structures that are both collections of data are
 - Sequential collections → indexed things such as arrays
 - Associative collections → values are referenced by a string key that maps it to a value, in JS is an object

```
let student = {};  
  
student.name = "Jake";           // or student["name"]  
student.age = 25;  
student.country = "Australia";  
  
console.log(student);  
  
console.log(Object.keys(student));  
// OUTPUT: [ "name", "age", "country" ]
```

```
console.log(Object.values(student));  
// OUTPUT: [ "Jake", 25, "Australia" ]
```

```
console.log(Object.entries(student));  
// OUTPUT: [  
//   ["name", "Jake"],  
//   ["age", 25],  
//   ["country", "Australia"]  
// ]
```

- Arrays and objects have methods
- Objects are the only non-primitive type

Chapter 2

Git

To effectively work on large projects with multiple people, we need advanced methods of managing the code-base. Simple file sharing is insufficient.

Git is a program that allows distributed version control where each user has a full backup. Work can also be stored in the cloud. GitHub, GitLab, and BitBucket are software tools that implement git via a web app (COMP1531 uses GitLab).

Git can be used as

1. Version control on a single machine
2. Version control across multiple of your machines
3. Version control across a team of engineers

`git diff` shows the difference between the working directory and staged directory.

Chapter 3

Unigotchi — Iteration 0

- Understand the requirements
- Design the function interfaces
- Set up how the team works together
- Write stub functions only (no mechanics)

1. Requirements
2. Design
3. Stub
4. MR
5. Review
6. Merge

Chapter 4

Importing Libraries

Libraries allow us to use code that other people have written in our own projects. In JavaScript, we can do so by adding the header `import fs from 'fs'`. Unlike C, the functions that are important can not be directly called. Instead, the functions exist as methods on objects, where the object name is specified when the library is imported.

We can also import our own code in other files. The functions that we want to use elsewhere can be exported by adding `export` or `export default`. The `default` keyword means that when the function is imported it will run the defaulted command. This is useful when providing an alias for the function.

Functions from other people can be downloaded into our directory using npm. Running `npm init` will create a `package.json` that has the information required modules to tell other people what to install. Installed packages are added to the `node_modules` folder. This folder is gitignored to save space.

Chapter 5

asdf

function requirements are the important parts non function are optional

Chapter 6

Dynamic Verification

- Writing tests are critical to ensure an application works, behaves as expected, and meets requirements
- Approaching testing the right way will yield better results

Types of software testing

- Acceptance
- Integration
- Unit
- Usability
- Functional
- Performance

6.0.1 Coverage

- A general measure of how thorough our tests are
- Test coverage is how much of the feature set is covered with tests
- Code coverage measure the percentage of lines of code that is covered in tests

Chapter 7

Static Verification

- TypeScript is similar to JavaScript but with explicit typing

Chapter 8

Continuous Integration

Practice of automating the integration of code changes into master. Typically continuous integration consists of a series of operations that are executed on any commit that is pushed to the repository. In gitlab, the `.gitlab-ci.yml` file within the root folder allows users to create tests that are run in stages before code can be merged onto master.

Chapter 9

Advanced Functions

- Writing long code causes longer loading times
- Since JavaScript is implemented on the web, the size of the files should be minimised.

```
// The traditional way of writing a function
function Volume(length, width, height) {
    return length * width * height;
}

// The same function can be written
const volume = function (length, width, height) => {
    return length * width * height;
}

// and more succinctly
const volume = (length, width, height) => {
    return length * width * height;
}

// condensed into one line
const volume = (length, width, height) => length * width * height;
```

- In JavaScript, functions are treated like variables, they can be stored, passed, or returned.
 - First class function
- A higher class function is able to return a function

```
type Fmrt = (str: string) => string;
const bracket = (str: string) => `${str}`;
const fullstop = (str: string) => `${str}.`;

function SayHi(name: string, format: Fmtr) {
    return `Hi ${format(name)}`;
}

console.log(SayHi("Name"), fullstop)
```

9.1 Map, Reduce, Filter

- Are array methods that perform iterative functions
- Map: Modify an array
 - Takes an array of size n , and produces a new array of size n having modified all elements of the array.
- Filter: Select from an array
 - Takes an array of size n and produces a new array of size 0 - n
- Reduce: Summarise an array

Chapter 10

HTTP Servers

- Web servers are a fundamental part of web-based full-stack software
- Network: A group of interconnected computers that can communicate
- Internet: A global infrastructure for networking computers around the world
- Nodejs express allows users to run a server locally

```
import express, { Request, Response } from 'express';

// Create an instance
const app = express();
const port: number = 3000;

app.use(express.text());

app.get('whatsup', (req: Request, res: Response) => {
  res.send('Hello!');
})

app.listen(port, () => {
  console.log('Listening on port ${port}');
})
```

Chapter 11

Persistence

Why data persistence?

- Persistence allows information to be retained even after the program stops running.
- Ensures consistency and reliability across multiple sessions or users.
- Data is also critical to allow an application to function as it allows for the storage of state.

Data layers

- In-memory (non-persistent)
- In-file (filesystem direct storage)
- In-database (Relational SQL, NoSQL)

Chapter 12

Exceptions

Chapter 13

Authentication and Authorisation

- Authentication → the process of verifying who a user is
- Authorisation → the process of determining what an authenticated user is allowed to do
- To keep passwords safe, we use hashing, a one way encryption method
- Encryption is hashing with a key so it can be decrypted

Chapter 14

Session

- Simple json tokens are easy to take
- Hash them with smth

Chapter 15

Modelling

- A model is a simplified representation of a more complex system that helps us understand, communicate, or reason about that system
- Conceptual model is a type of model that captures a system in a conceptual rather than physical model, tending to be diagrammatic or visual
- Behavioural model
 - Distinct states a system can be in
 - Transitions between states triggered by events/actions
 - Some variation in notation, typically states are circles, transitions are labelled arrows

Chapter 16

Software Complexity

- Essential
 - Complexity that is inherent to the problem
 - Eg. if the user or client requires the program to do 30 different things, then those 30 things are essential
- Accidental
 - Complexity that is not inherent to the problem
 - Eg. generating or parsing data in specific formats
- Coupling
 - A measure of how closely connected different software components are
 - Usually expressed as a simple ordinal measure of 'loose' or 'tight'
 - Eg. web applications tend to have a front-end that is loosely coupled from the backend
 - Loose coupling is good
- Cohesion
 - The degree to which elements of a module belong together
 - Elements belong together if they are somehow related
 - High cohesion is good
- Cyclomatic complexity
 - A measure of the branching complexity of functions
 - Computed by counting the number of linearly-independent paths through a function
 - Convert function into a control flow graph

$$V(G) = e - n + 2$$

where e is the number of edges and n is the number of nodes.

- Low cyclomatic complexity is better

Chapter 17

Agile Software Engineering

- What is agile?
 - Agile is a software development philosophy that focuses on delivering valuable software through small, frequent releases rather than one large release at the end of a project
 - Unlike traditional approaches, Agile assumes that requirements will change as customers learn more about their needs
 - The goal is not simply writing code, but continuously delivering value to users
 - In 1990s many software projects failed because teams spent months gathering requirements, writing documentation, and producing detailed designs before writing code
 - Customer needs changed
 - Competitors released better products faster
 - Agile practices
 - Build small pieces of working software
 - Release frequently
 - Gather customer feedback early
 - Adapt to changing requirements
 - Continuously improve the product
1. Satisfy customers through early delivery
 2. Welcome changing requirements
 3. Deliver software frequently
 4. Developers and customers work together
 5. Build projects around motivated people
 6. Face-to-face conversations are the most effective way to share information
 7. Working software is the primary measure of progress
 8. Maintain sustainable development
 9. Pursue technical excellence
 10. Keep solutions simple

- **Key Agile Values**
 - * Individuals and interactions over processes and tools
 - * Working software over comprehensive documentation
 - * Customer collaboration over contract negotiation
 - * Responding to change over following a plan
- Successful Agile teams commonly use
 - * User stories
 - * Test driven development
- Why agile works
 - Agile succeeds because it shortens the feedback loop
 - Fast feedback means fewer mistakes
- Some examples

17.1 Validation

- It's important to understand the difference between verification and validation
- Validation in a system life cycle context is a set of activities ensuring and gaining confidence that a system is able to accomplish its intended use, goals and objectives.

17.2 Deployment